

BRUTALLY HONEST

Developer Best Practices Guide

The mistakes we see again and again. From juniors and 5-year seniors alike. SQL · Security · APIs · Performance · Architecture · Production

By **Sadek Hossain**

Founder & CEO, EzyCourse

f facebook.com/sadek.hkm.hossain

in linkedin.com/in/hkm-hossain

01	Never Trust the Client Input validation, sanitisation & user-friendly errors
02	Error Handling — UX vs Security Field errors, global 500 handler, error tracking
03	Auth vs Authorisation & IDOR Logged in ≠ allowed. Ownership checks. Multi-tenant.
04	SQL Indexes & Query Performance The most skipped topic. Unique indexes. Selectivity.
05	The N+1 Query Problem SQL JOIN, Sequelize eager load, MongoDB populate
06	Loading Everything at Once Pagination, unbounded queries, cursor vs offset
07	Resource Limits & Rate Limiting Credit enforcement, brute force, Cloudflare/AWS WAF
08	Duplicate API Calls Loading states, AbortController, useEffect pitfalls
09	Return Only What's Needed Never expose sensitive fields. Treat user data as gold.
10	Password Storage & Exposed Secrets bcrypt, env vars, git exposure
11	Race Conditions SELECT FOR UPDATE, atomic updates, Promise.all

12	Parallel API & DB Calls Promise.all, why sequential calls are wasteful
13	File Upload Security Always use third-party storage. Never same server.
14	Production Architecture Proxy servers, CDN, never expose origin directly
15	Logging & Error Monitoring Bugsnap, Sentry, Slack alerts — fly with instruments
16	Simulate Scale Locally Seed 1M rows, k6 load test, slow query log
17	Quick Reference Rules to paste on your wall

— 01 —

Never Trust the Client

Validate everything server-side. Return clear, friendly, field-level error messages.

Frontend validation is **UX only**. Anyone with curl or Postman bypasses it in seconds. Your backend must validate independently — and when it fails, return **specific, user-friendly messages per field**. Not 'Validation error'. Not a 422 with no body. Tell the user exactly what is wrong.

■ Vague, useless error response

```
app.post('/register', async (req, res) => {
  if (!req.body.email || !req.body.password) {
    return res.status(422).json({
      error: 'Validation error' // useless ■
    });
  }
  // Also: no type check, no length check
  // age: -999 accepted
  // role: 'admin' accepted from client ■
  await User.create(req.body);
});
```

✓ Field-level friendly errors

```
// Using Zod (works with any framework)
const schema = z.object({
  email: z.string().email('Enter a valid email'),
  password: z.string().min(8, 'Password must be 8+ chars'),
  age: z.number().int().min(13, 'Must be 13+').max(120),
  // role never accepted from client – always set server-side
});
const result = schema.safeParse(req.body);
if (!result.success) {
  return res.status(422).json({
    errors: result.error.flatten().fieldErrors
    // { email: ['Enter a valid email'], ... }
  });
}
```

Also validate:

- **String lengths** — a 10MB 'username' will overflow your DB column
- **Array sizes** — req.body.items with 50,000 entries will eat your RAM
- **Data types** — is that actually a number, or the string '999999999999'?
- **Mass assignment** — never do User.create(req.body); always whitelist fields
- **File types** — check MIME type server-side, not just the file extension

Mass assignment: if you pass req.body directly to your ORM, a user can set role: 'admin' or credits: 999999 just by including it in the POST body. Always explicitly pick the fields you accept.

— 02 —

Error Handling — UX vs Security

Field errors for users. Generic 500s to clients. Full details to your logging system.

There are two types of errors: **expected errors** (validation, not found, unauthorised) and **unexpected errors** (crashes, unhandled exceptions). Expected errors get friendly messages. Unexpected errors get a generic 'Something went wrong' to the client — and full stack trace + context to your error monitoring tool.

✓ Global error handler — the last line of defence

```
// Express — add this AFTER all routes
app.use((err, req, res, next) => {
  // Log everything internally with context
  logger.error({
    message: err.message,
    stack: err.stack,
    url: req.url,
    method: req.method,
    userId: req.user?.id,
    body: req.body, // careful: redact passwords/tokens
    // send to Bugsnag / Sentry automatically via their SDK
  });

  // Send NOTHING useful to the client
  if (err.isOperational) {
    // Known errors: 404, 403, validation etc
    return res.status(err.statusCode).json({ error: err.message });
  }

  // Unknown crash — generic only
  res.status(500).json({ error: 'Something went wrong. We have been notified.' });
});
```

■ Never expose internals to the client in production

```
// What many devs accidentally ship:
app.use((err, req, res, next) => {
  res.status(500).json({
    error: err.message, // leaks DB structure, file paths
    stack: err.stack, // leaks your entire codebase layout ■
  });
});

// Error message example that leaks too much:
// 'column users.pasword does not exist at character 40'
// Attacker now knows your table name, column names, and that you have a typo.
```

Recommended error monitoring tools:

- **Bugsnag** — instant Slack + email alerts, grouping, user context. Highly recommended.
- **Sentry** — open source option, strong JS/Node support
- **Datadog** — full observability if you need metrics + logs + traces
- All of these integrate in under 10 minutes and notify you the moment something breaks in prod

You should know about a production crash before your users do. If you are finding out from a user complaint — you have no error monitoring. Fix that first.

— 03 —

Auth vs Authorisation & IDOR

Being logged in does not mean you can access everything. Check ownership. Always.

Authentication = who are you (login, JWT, session). **Authorisation** = are you allowed to do this specific thing. Most developers implement authentication correctly. Many skip ownership checks entirely, creating IDOR vulnerabilities.

IDOR — Insecure Direct Object Reference

IDOR happens when you fetch a record by its ID without checking whether the requesting user owns it. An attacker changes the ID in the URL and reads other users' data.

■ Classic IDOR — ID only, no ownership check

```
// GET /api/invoices/1042 - yours
// GET /api/invoices/1043 - someone else's
// GET /api/invoices/1 - first ever invoice
app.get('/invoices/:id', auth, async (req, res) => {
  const invoice = await Invoice.findByPk(
    req.params.id // no ownership check ■
  );
  res.json(invoice);
  // Any logged-in user reads any invoice
});
```

✓ Scoped to authenticated user

```
app.get('/invoices/:id', auth, async (req, res) => {
  const invoice = await Invoice.findOne({
    where: {
      id: req.params.id,
      userId: req.user.id // ownership ■
    }
  });
  if (!invoice) {
    // 404 not 403 - don't confirm it exists
    return res.status(404).json({ error: 'Not found' });
  }
  res.json(invoice);
});
```

Multi-tenant systems — add tenant check too

✓ Must scope to both user AND tenant in SaaS products

```
// Bad: scoped to user only
await Invoice.findOne({ where: { id, userId: req.user.id } });

// Good: scoped to user AND tenant (org/workspace/account)
await Invoice.findOne({
  where: {
    id: req.params.id,
    userId: req.user.id,
    tenantId: req.user.tenantId, // prevents cross-tenant access ■
  }
});

// For sub-resources (e.g. comment belongs to post belongs to user):
// If you can't directly check userId on the comment table,
// JOIN up to the parent and check ownership there
await Comment.findOne({
  where: { id: commentId },
  include: [{ model: Post, where: { userId: req.user.id } }]
  // If post doesn't belong to user: returns null -> 404 ■
});
```

Rule: if you are fetching a record by ID alone with no userId or tenantId condition, you almost certainly have an IDOR. Every query on user-owned data must be scoped.

— 04 —

SQL Indexes & Query Performance

Don't wait for 10K rows. Add indexes from day one — tables grow faster than you think.

An index is a separate data structure your DB maintains so it can find rows without scanning every single one. The common advice is 'add indexes when you have enough rows.' **Wrong.** Add indexes when you write the query. Tables grow unnoticed. By the time you feel the pain, your users already have.

■ No index — full table scan

```
-- users: 2 million rows, no index on email
SELECT * FROM users
WHERE email = 'alex@example.com';

-- DB reads ALL 2M rows
-- Time: ~4200ms ■
-- EXPLAIN shows: Seq Scan
```

✓ With index — direct lookup

```
CREATE INDEX idx_users_email
ON users(email);

SELECT * FROM users
WHERE email = 'alex@example.com';

-- DB jumps directly to the row
-- Time: ~0.8ms ■
-- EXPLAIN shows: Index Scan
```

UNIQUE indexes — use them, not just for speed but correctness

A UNIQUE index prevents duplicate values AND gives you a fast lookup. Use it on columns that must be unique: email, username, slug, API key. Without it, two concurrent registrations with the same email can both succeed if your application-level check has a race condition.

✓ UNIQUE index examples

```
-- Prevents duplicate emails AND creates a fast index
ALTER TABLE users ADD CONSTRAINT uq_users_email UNIQUE (email);

-- Prevents duplicate slug per user (composite unique)
ALTER TABLE posts ADD CONSTRAINT uq_posts_user_slug UNIQUE (user_id, slug);

-- Prevents same user liking a post twice
ALTER TABLE likes ADD CONSTRAINT uq_likes_user_post UNIQUE (user_id, post_id);

-- All of the above also serve as indexes — no extra CREATE INDEX needed
```

Selectivity — not all columns deserve an index

Selectivity = how unique the values are. A boolean or status column with 2 values has terrible selectivity — the DB may ignore the index entirely. Combine low-selectivity with high-selectivity columns in a composite index.

✓ Composite index — combine for better selectivity

```
-- 'status' alone: only 2 values, low selectivity
-- 'community_id' alone: moderate
-- Together in a composite: highly selective ■
CREATE INDEX idx_posts_community_status_date
ON posts(community_id, status, created_at DESC);

-- Covers this query perfectly:
SELECT * FROM posts
WHERE community_id = 5 AND status = 'published'
ORDER BY created_at DESC LIMIT 20;

-- Always verify with EXPLAIN ANALYZE:
-- Look for 'Index Scan' not 'Seq Scan'
```

Don't index everything. Each index slows down INSERT/UPDATE/DELETE because the index must be updated too. Index what you query. But for any column in a WHERE or JOIN — add it from day one.

— 05 —

The N+1 Query Problem

500 posts. 1 query to fetch them. Then 500 more to get each author. This is N+1.

N+1 is invisible in development with 5 records. In production with 500 it makes 501 database calls per page load. The fix is always: fetch related data in the same query using JOIN or eager loading.

SQL / Sequelize (Node.js)

■ 501 DB calls for 500 posts

```
const posts = await Post.findAll();
const result = await Promise.all(
  posts.map(async (post) => {
    // 1 DB query per post ■
    const author = await User
      .findByPk(post.userId);
    return { ...post.toJSON(), author };
  })
);
// 500 posts = 501 queries
```

✓ 1 query with eager loading

```
const posts = await Post.findAll({
  include: [{
    model: User,
    as: 'author',
    attributes: ['id', 'name', 'avatar']
    // only select what you need ■
  }],
  order: [['createdAt', 'DESC']],
  limit: 20
});
// 1 JOIN query. All data. Done. ■
```

MongoDB / Mongoose (Node.js)

■ N+1 in MongoDB

```
const posts = await Post.find({})
  .limit(20);
// 1 query per post for author ■
for (const post of posts) {
  post.author = await User.findById(
    post.userId
  );
}
// 21 DB calls for 20 posts
```

✓ populate() — 1 extra query total

```
const posts = await Post
  .find({})
  .populate({
    path: 'userId',
    select: 'name avatar', // fields only ■
    as: 'author'
  })
  .limit(20)
  .lean(); // plain JS objects, faster
// 2 queries total regardless of count ■
```

✓ Raw SQL when ORM is not enough

```
-- Single query fetches posts + author + like count
SELECT
  p.id, p.body, p.created_at,
  u.id AS author_id,
  u.name AS author_name,
  u.avatar AS author_avatar,
  COUNT(l.id) AS like_count
FROM posts p
JOIN users u ON u.id = p.user_id
LEFT JOIN likes l ON l.post_id = p.id
WHERE p.community_id = :communityId
GROUP BY p.id, u.id
ORDER BY p.created_at DESC
LIMIT 20;

-- One round-trip. All data. No ORM magic needed.
```

How to catch it: enable SQL query logging. If you see the same query repeated with different IDs — that is N+1. Fix before merging.

— 06 —

Loading Everything at Once

Your local DB has 20 rows. Production has 2 million. Plan for that now.

Fetching all records in a table is the most common cause of out-of-memory crashes in production APIs. It works in dev. It kills servers at scale. Paginate everything — posts, comments, replies, members, search results.

■ Loads entire table into RAM

```
app.get('/posts', async (req, res) => {
  const posts = await Post.findAll({
    where: { communityId: req.params.id }
  });
  // 80,000 posts * 2KB = 160MB per request
  // Multiple users = server dead ■
  res.json(posts);
});
```

✓ Cursor-based pagination

```
app.get('/posts', async (req, res) => {
  const { cursor, limit = 20 } = req.query;
  const cap = Math.min(Number(limit), 50);
  const posts = await Post.findAll({
    where: {
      communityId: req.query.communityId,
      ...(cursor && { id: { [Op.lt]: cursor } })
    },
    order: [['id', 'DESC']],
    limit: cap
  });
  res.json({ posts, next: posts.at(-1)?.id });
});
```

■ Why OFFSET fails — DB scans and discards rows

```
-- Page 1: fast. Reads rows 1-20.
SELECT * FROM posts ORDER BY id DESC LIMIT 20 OFFSET 0; -- 1ms

-- Page 5000: DB reads AND THROWS AWAY 100,000 rows, then returns 20
SELECT * FROM posts ORDER BY id DESC LIMIT 20 OFFSET 100000; -- 900ms ■

-- Cursor-based: always fast, regardless of depth
SELECT * FROM posts WHERE id < 500000 ORDER BY id DESC LIMIT 20; -- 1ms ■
```

Hard rule: no endpoint ever returns unbounded rows. Add a hard maximum LIMIT server-side even when the client doesn't send one. Never trust the client to paginate themselves.

— 07 —

Resource Limits & Rate Limiting

Two different things. Both must live on the server. Both can be enforced at the edge too.

Resource limits = does this user have enough credits/quota to perform this action. **Rate limiting** = is this IP/user sending too many requests too fast (brute force, abuse). These are different problems. Both must be enforced server-side. Frontend checks are UX only — anyone bypasses them with a direct API call.

Resource / Credit Checking

■ Frontend-only credit check

```
// React — completely bypassable
const generate = () => {
  if (credits <= 0) {
    toast('No credits');
    return; // attacker ignores this
  }
  callAPI('/generate', prompt);
  // POST /generate called directly
  // skips this check entirely ■
};
```

✓ Server enforces credit check

```
app.post('/generate', auth, async (req, res) => {
  const user = await User.findByPk(req.user.id);
  if (user.credits < COST) {
    return res.status(402).json({
      error: 'Insufficient credits'
    });
  }
  await user.decrement('credits', { by: COST });
  const result = await callAI(req.body.prompt);
  res.json(result); // ■
});
```

Rate Limiting — brute force, spam & abuse prevention

✓ Rate limit login, register, OTP — any sensitive endpoint

```
// express-rate-limit
import rateLimit from 'express-rate-limit';

const loginLimiter = rateLimit({
  windowMs: 15 * 60 * 1000, // 15 minutes
  max: 10, // 10 attempts per window per IP
  message: { error: 'Too many attempts. Try again later.' },
  standardHeaders: true,
  legacyHeaders: false,
});

app.post('/login', loginLimiter, loginHandler);
app.post('/register', loginLimiter, registerHandler);
app.post('/forgot-password', loginLimiter, forgotHandler);

// Without this: attacker tries 100,000 passwords/min ■
```

Rate limiting at the edge — before it hits your server

The best rate limiting happens before a request reaches your application server. Use Cloudflare or AWS WAF/Shield to block abuse at the network layer.

■ Cloudflare Rate Limiting rule (set in dashboard or via API)

```
# Cloudflare Rate Limiting — Dashboard > Security > WAF > Rate Limiting
#
# Rule: Block IPs that hit /api/login more than 20 times in 1 minute
# Expression: (http.request.uri.path eq "/api/login")
# Threshold: 20 requests / 60 seconds / per IP
# Action: Block (or JS Challenge for suspicious traffic)
#
# AWS WAF equivalent:
# Create a Rate-based rule in AWS WAF attached to your ALB/CloudFront
# Threshold: 100 requests / 5 minutes per IP
# Action: Block
#
# This stops attackers before they ever reach Node.js / your DB
```

Layer your rate limiting: Cloudflare/AWS at the edge for volumetric attacks, application-level (express-rate-limit) for business logic. Edge protection is cheaper — it doesn't consume your server resources at all.

— 08 —

Duplicate API Calls

One action = one request. React makes this easier to get wrong than right.

Duplicate calls create duplicate data, double charges, and unnecessary server load. They come from: double-clicks with no loading state, React `useEffect` firing twice (StrictMode + dependency bugs), event listeners attached more than once, and GET calls triggered by multiple state changes at component mount.

POST/mutation calls — always use loading state

■ No guard — double submit

```
function PayButton() {
  const handleClick = async () => {
    // User clicks twice = 2 charges ■
    await api.post('/orders', cart);
    router.push('/success');
  };
  return (
    <button onClick={handleClick}>
      Pay Now
    </button>
  );
}
```

✓ Loading state + disabled

```
function PayButton() {
  const [loading, setLoading] = useState(false);
  const handleClick = async () => {
    if (loading) return;
    setLoading(true);
    try {
      await api.post('/orders', cart);
      router.push('/success');
    } finally {
      setLoading(false);
    }
  };
  return (
    <button onClick={handleClick} disabled={loading}>
      {loading ? 'Processing...' : 'Pay Now'}
    </button>
  ); // ■
}
```

GET calls in `useEffect` — StrictMode double fire & dependency bugs

■ useEffect fires twice in development (StrictMode) — and sometimes in production too

```
// Fires twice in dev (React StrictMode mounts, unmounts, remounts)
// Can fire multiple times in prod if dependencies change rapidly
useEffect(() => {
  fetchUserPosts(userId); // 2-3 API calls ■
}, [userId, filter, page, sortOrder]);
// Every time ANY dependency changes, another API call fires
// If parent re-renders frequently, this hammers your server
```

✓ Fix: AbortController cancels stale requests

```
useEffect(() => {
  const controller = new AbortController();

  const fetchData = async () => {
    try {
      const res = await fetch(`/api/posts?userId=${userId}`, {
        signal: controller.signal
      });
      const data = await res.json();
      setPosts(data);
    } catch (err) {
      if (err.name === 'AbortError') return; // cleanup, not a real error
      setError(err);
    }
  };

  fetchData();
  return () => controller.abort(); // cancel on re-render or unmount ■
}, [userId]);
```

Also consider: React Query or SWR for data fetching. They deduplicate identical requests automatically, cache results, and handle loading/error states — solving most of these problems out of the box.

— 09 —

Return Only What's Needed

Never expose sensitive fields. Treat user data like a treasure — only the owner sees it.

Every API response is a potential data leak. When you fetch posts with their authors, you needed the author's name and avatar — not their email, phone number, hashed password, or internal flags. Returning extra fields is not harmless. It exposes private user data to everyone who can call that endpoint.

■ Returns everything — leaks email to all

```
// Fetching posts with author
const posts = await Post.findAll({
  include: [{
    model: User,
    as: 'author'
  }]
  // No attributes filter = returns:
  // name, email, phone, password,
  // role, createdAt, stripeId... ■
});
// Anyone who can see the post
// now sees the author's email
```

✓ Explicitly select safe fields only

```
const posts = await Post.findAll({
  include: [{
    model: User,
    as: 'author',
    attributes: ['id', 'name', 'avatar']
    // Only public-safe fields ■
    // email never leaves the server
  }]
});
// MongoDB equivalent:
Post.find({}).populate('userId',
  'name avatar' // field selection ■
);
```

■ Fields that should NEVER appear in public API responses

```
// These should only ever be visible to the account owner,
// or never in API responses at all:

password // hashed or not — never ever
email // only visible to the owner themselves
phone // same — owner only
stripeCustomerId, paymentMethodId // payment internals
role, permissions // internal flags not needed by frontend
resetToken, verifyToken // security tokens
internalNotes, adminFlags // internal metadata

// Treat email like a bank account number.
// It belongs to the user. Not to anyone who views their profile.
```

For your own profile endpoint vs public profile:

✓ Return different shapes depending on who is asking

```
app.get('/users/:id', auth, async (req, res) => {
  const isOwner = req.user.id === req.params.id;
  const user = await User.findByPk(req.params.id, {
    attributes: isOwner
      ? ['id', 'name', 'avatar', 'email', 'phone', 'createdAt']
      : ['id', 'name', 'avatar'], // public: name and avatar only
  });
  res.json(user); // ■ owner sees more, others see less
});
```

Default to returning the minimum. Add fields when there is a specific need. Never default to returning everything and removing later — by then it's already in the wild.

— 10 —

Password Storage & Exposed Secrets

Still the most catastrophic mistakes — and still shockingly common.

Storing passwords in plain text or MD5 means that if your database is ever breached, every user's password is immediately exposed. API keys hardcoded in source code and pushed to GitHub are found by bots within minutes.

■ Never — plain text or MD5

```
// Plain text ■
await User.create({
  email: req.body.email,
  password: req.body.password
});

// MD5 — also wrong ■
const hash = crypto
  .createHash('md5')
  .update(password)
  .digest('hex');

// Cracked in milliseconds
// with rainbow tables
```

✓ bcrypt — salted, slow, correct

```
import bcrypt from 'bcrypt';
const ROUNDS = 12; // minimum 10

// Register:
const hash = await bcrypt.hash(password, ROUNDS);
await User.create({ email, password: hash });

// Login:
const valid = await bcrypt.compare(
  req.body.password,
  user.password
);
if (!valid) throw new AuthError(); // ■
```

■ Never hardcode secrets — use environment variables

```
// Seen in real production repos every week
const stripe = new Stripe('sk_live_4eC39HqLyjWDarjtT1zdp7dc'); // ■
const db = mysql.createConnection({ password: 'MyProd123!' }); // ■
const jwt = sign(payload, 'mysecret'); // ■

// Once pushed to git: it is compromised FOREVER.
// Even if you delete the commit. Git history is permanent.
// Bots scan all public repos 24/7 specifically for these patterns.

// Fix:
const stripe = new Stripe(process.env.STRIPE_SECRET_KEY);
// .env file in .gitignore — never committed
```

If you accidentally push a secret key: rotate it IMMEDIATELY — before you even write the fix commit. The key is compromised the moment it appears in a public repo.

— 11 —

Race Conditions

Two requests. Same data. One broken state. Impossible to reproduce. Easy to prevent.

A race condition happens when two requests execute simultaneously and both assume exclusive access to the same row. Classic case: credit deduction. Both read 10 credits. Both pass the check. Both deduct. You end up at -10.

■ Race condition — credits deducted twice

```
// Request A and B arrive at the same millisecond
async function useCredits(userId, cost) {
  const user = await User.findByPk(userId); // A reads: 10 credits
  // B reads: 10 credits (stale!)
  if (user.credits < cost) throw new Error('Not enough');
  // Both A and B pass this check

  await user.update({ credits: user.credits - cost });
  // A writes: 10 - 10 = 0
  // B writes: 10 - 10 = 0 (used stale value - B should have seen 0!)
  // Result: service used twice. Paid for once. ■
}
```

✓ Fix: atomic update or SELECT FOR UPDATE

```
// Option 1: Atomic SQL – simplest and fastest
const [updated] = await db.query(
  'UPDATE users SET credits = credits - ? WHERE id = ? AND credits >= ?',
  [cost, userId, cost]
);
if (updated === 0) throw new Error('Insufficient credits'); // 0 rows = failed

// Option 2: Transaction with row lock (SELECT FOR UPDATE)
await sequelize.transaction(async (t) => {
  const user = await User.findOne({
    where: { id: userId },
    lock: t.LOCK.UPDATE, // locks this row until transaction commits
    transaction: t
  });
  if (user.credits < cost) throw new Error('Insufficient credits');
  await user.decrement('credits', { by: cost, transaction: t });
});
// B now waits for A to finish – no stale read ■
```

Race conditions also hit: inventory (overselling), referral bonuses (claimed twice), unique username registration, and appointment slots. Any time two users can act on the same resource simultaneously — use a lock or atomic update.

— 12 —

Parallel API & DB Calls

Independent operations run simultaneously. Sequential calls are wasted time.

When you have multiple independent operations — fetch user, fetch posts, call an external API — most developers write them sequentially. Each waits for the previous to finish. If each takes 100ms, 5 sequential calls = 500ms total. Run them in parallel: 5 calls still finishes in ~100ms.

■ Sequential — 300ms total

```
// Each awaits the previous one
// Total time: 100 + 100 + 100 = 300ms
const user = await getUser(id); // 100ms
const posts = await getUserPosts(id); // 100ms
const followers = await getFollowers(id); // 100ms

// None of these depend on each other
// But each waits for the last to finish ■
```

✓ Parallel — ~100ms total

```
// All fire at the same time
// Total time: max(100, 100, 100) = ~100ms
const [user, posts, followers] = await Promise.all([
  getUser(id), // fires immediately
  getUserPosts(id), // fires immediately
  getFollowers(id), // fires immediately
]);
// 3x faster for the same result ■
```

✓ Real-world dashboard endpoint — 5 independent queries

```
app.get('/dashboard', auth, async (req, res) => {
  const uid = req.user.id;

  // All independent – run in parallel
  const [user, posts, notifications, stats, billing] = await Promise.all([
    User.findByPk(uid, { attributes: ['id', 'name', 'avatar'] }),
    Post.count({ where: { userId: uid } }),
    Notification.count({ where: { userId: uid, read: false } }),
    Analytics.getSummary(uid), // external API call
    BillingService.getStatus(uid), // another external call
  ]);

  // Sequential this would be: 100+80+60+200+150 = 590ms
  // Parallel: max(200ms) = ~200ms ■

  res.json({ user, posts, notifications, stats, billing });
});
```

■ Promise.allSettled — when you want results even if some fail

```
// Promise.all fails EVERYTHING if ONE rejects
// Use allSettled when partial results are acceptable
const results = await Promise.allSettled([
  getUser(id),
  getAnalytics(id), // if this fails, we still want the rest
  getRecommendations(id),
]);

const [userResult, analyticsResult, recsResult] = results;
const user = userResult.status === 'fulfilled' ? userResult.value : null;
const analytics = analyticsResult.status === 'fulfilled' ? analyticsResult.value : null;
```

Rule: if operation B does not need the result of operation A — run them in parallel. Sequential code is the path of least resistance but it is almost always slower than it needs to be.

— 13 —

File Upload Security

Never store uploads on your own server. Use S3, DO Spaces, Bunny, or similar. Always.

Storing uploaded files on your application server is a mistake on every level: it is a security vulnerability, it doesn't scale, and it ties your storage to your compute. If you ever need two servers, your uploads disappear. Always use dedicated object storage. Always.

■ Dangerous: files stored on app server, client controls filename

```
app.post('/upload', upload.single('file'), (req, res) => {
  const filename = req.file.originalname; // attacker controls this ■
  fs.writeFileSync('./public/uploads/' + filename, req.file.buffer);
  // Attacker uploads: 'shell.php' → yoursite.com/uploads/shell.php → execute code
  // Attacker uploads: '../../../config/db.js' → overwrites your config
  // Attacker uploads: 'evil.html' → runs scripts in your app cookie context
  res.json({ url: '/uploads/' + filename });
});
```


✓ Correct: validate, rename, upload to object storage

```
import { S3Client, PutObjectCommand } from '@aws-sdk/client-s3';
import { v4 as uuid } from 'uuid';

const s3 = new S3Client({ region: process.env.AWS_REGION });

app.post('/upload', auth, upload.single('file'), async (req, res) => {
  const file = req.file;

  // 1. Validate MIME type server-side (not just extension)
  const allowed = ['image/jpeg', 'image/png', 'image/webp', 'image/gif'];
  if (!allowed.includes(file.mimetype)) {
    return res.status(400).json({ error: 'File type not allowed' });
  }

  // 2. Enforce size (also set limit in multer config)
  if (file.size > 5 * 1024 * 1024) { // 5MB
    return res.status(400).json({ error: 'Max file size is 5MB' });
  }

  // 3. Generate safe filename — never trust client filename
  const extensions = { 'image/jpeg': 'jpg', 'image/png': 'png',
    'image/webp': 'webp', 'image/gif': 'gif' };
  const key = `uploads/${req.user.id}/${uuid()}.${extensions[file.mimetype]}`;

  // 4. Upload to S3 (or D0 Spaces, Bunny, Cloudflare R2)
  await s3.send(new PutObjectCommand({
    Bucket: process.env.S3_BUCKET,
    Key: key,
    Body: file.buffer,
    ContentType: file.mimetype,
  }));

  res.json({ url: `https://cdn.yourdomain.com/${key}` }); // CDN URL ■
});
```

Good options for object storage:

- **AWS S3** — industry standard, most integrations, globally available
- **DigitalOcean Spaces** — S3-compatible, simpler pricing, good for smaller teams
- **BunnyCDN Storage** — extremely fast CDN + storage, competitive pricing
- **Cloudflare R2** — zero egress fees, S3-compatible

All of them give you a CDN URL, take the load off your server, and scale infinitely.

Never serve uploaded files from the same domain as your app. An uploaded HTML or SVG file can execute JavaScript in your app's cookie context, stealing sessions of anyone who views it.

— 14 —

Production Architecture

Never expose your origin server directly. Always put a proxy in front.

In production, your application server should never be directly reachable from the internet. A proxy layer in front handles SSL termination, DDoS protection, caching, and hides your server's real IP. Without it, attackers target your origin directly, bypassing all your CDN and firewall rules.

✓ Recommended production stack

```
Internet
|
v
Cloudflare (DNS + DDoS protection + WAF + Rate limiting + CDN)
|
v
AWS ALB / Nginx (Load balancer, SSL termination, reverse proxy)
|
v
App Servers (Node.js / your backend – private subnet, not public)
|
v
Database (RDS / Postgres – private subnet, no public access ever)

# Your app server's IP should NEVER appear in public DNS.
# Cloudflare proxies all traffic – origin IP stays hidden.
# Enable Cloudflare 'I'm Under Attack' mode during incidents.
```

Nginx as reverse proxy — basic config

■ Nginx in front of Node.js — never expose Node directly on port 80/443

```
# /etc/nginx/sites-available/app.conf
server {
    listen 443 ssl;
    server_name app.example.com;

    ssl_certificate /etc/letsencrypt/live/app.example.com/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/app.example.com/privkey.pem;

    # Security headers
    add_header X-Frame-Options DENY;
    add_header X-Content-Type-Options nosniff;
    add_header Referrer-Policy strict-origin-when-cross-origin;

    location /api {
        proxy_pass http://127.0.0.1:3000; # Node.js on localhost only
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    }
}

# Node.js binds to 127.0.0.1:3000 – never 0.0.0.0:3000
```

Your database must NEVER have a public IP. It should only accept connections from your app servers, on a private VPC subnet. One exposed DB port = instant data breach.

— 15 —

Logging & Error Monitoring

You should know your app is broken before your users do.

Console.log is not monitoring. If your only visibility into production is SSH + tail -f, you are flying blind. Error monitoring tools catch every unhandled exception, group them, show you the stack trace, the user affected, and alert you via Slack or email — instantly. Setup takes under 10 minutes.

✓ Bugsnag setup — 5 minutes, instant Slack alerts

```
// npm install @bugsnag/js @bugsnag/plugin-express
import Bugsnag from '@bugsnag/js';
import BugsnagPluginExpress from '@bugsnag/plugin-express';

Bugsnag.start({
  apiKey: process.env.BUGSNAG_API_KEY,
  plugins: [BugsnagPluginExpress],
  releaseStage: process.env.NODE_ENV,
  enabledReleaseStages: ['production', 'staging'],
});

const middleware = Bugsnag.getPlugin('express');
app.use(middleware.requestHandler); // before routes
// ... your routes ...
app.use(middleware.errorHandler); // after routes

// That's it. Every unhandled error now goes to Bugsnag.
// Connect Bugsnag -> Slack in 2 clicks. Instant alerts. ■
```

■ Add user context — know exactly who was affected

```
// Tell Bugsnag who the user is on each request
app.use((req, res, next) => {
  if (req.user) {
    Bugsnag.setUser(req.user.id, req.user.email, req.user.name);
  }
  next();
});

// Add custom data to errors:
Bugsnag.notify(new Error('Payment failed'), (event) => {
  event.addMetadata('payment', {
    amount: charge.amount,
    currency: charge.currency,
    stripeError: charge.error,
  });
});
```

Recommended tools:

- **Bugsnag** — best Slack/email integration, great grouping, easy setup. Recommended.
 - **Sentry** — open source option, strong JavaScript support, self-hostable
 - **Datadog** — full APM, metrics, traces, logs in one platform (larger teams)
 - **LogRocket** — front-end session replay + error tracking
- Connect any of these to Slack and you will get a DM the moment something breaks in prod.

The question is not whether your production app will throw errors. It will. The question is whether you will know about it in 30 seconds or 3 days.

— 16 —

Simulate Scale Locally

You don't need a big company to learn big-system problems. Build the problems yourself.

Most developers never encounter scale problems at work because their companies never grow large enough. Here is how to manufacture the experience on your own laptop. Do this once and you will understand what 5 years of CRUD work cannot teach you.

Step 1 — Seed 1 million+ rows instantly

■ PostgreSQL generate_series — inserts millions of rows in seconds

```
-- 1 million users with realistic data distribution
INSERT INTO users (name, email, status, created_at)
SELECT
  'User ' || i,
  'user' || i || '@example.com',
  CASE WHEN random() > 0.1 THEN 'active' ELSE 'inactive' END,
  NOW() - (random() * INTERVAL '2 years')
FROM generate_series(1, 1000000) AS i;

-- 5 million posts across 100 communities
INSERT INTO posts (user_id, community_id, body, created_at)
SELECT
  (random() * 999999 + 1)::int,
  (random() * 99 + 1)::int,
  md5(random()::text),
  NOW() - (random() * INTERVAL '1 year')
FROM generate_series(1, 5000000) i;
```

Step 2 — Prove indexes with your own eyes

■ Before and after — you will never forget this after doing it once

```
-- BEFORE index:
EXPLAIN ANALYZE
SELECT * FROM posts WHERE community_id = 5 ORDER BY created_at DESC LIMIT 20;
-- Seq Scan on posts (actual time=890..3200ms) Rows: 5,000,000 scanned

-- Add the index:
CREATE INDEX idx_posts_community_date ON posts(community_id, created_at DESC);

-- AFTER index (same query):
-- Index Scan using idx_posts_community_date (actual time=0.1..0.9ms)
-- 3200ms → 0.9ms. Visible. Undeniable.
```

Step 3 — Load test with k6 (free)

■ brew install k6 or npm install -g k6

```
// load-test.js
import http from 'k6/http';
import { sleep, check } from 'k6';

export const options = {
  stages: [
    { duration: '30s', target: 50 }, // ramp to 50 concurrent users
    { duration: '1m', target: 200 }, // spike to 200
    { duration: '30s', target: 0 }, // wind down
  ],
};

export default function () {
  const res = http.get('http://localhost:3000/api/feed?communityId=5');
  check(res, { 'status 200': (r) => r.status === 200 });
  sleep(1);
}

// Run: k6 run load-test.js
// Watch memory, CPU, DB connections, response time. See what breaks.
```

Step 4 — Slow query log**■ Log every query over 200ms in PostgreSQL**

```
ALTER SYSTEM SET log_min_duration_statement = 200; -- ms
SELECT pg_reload_conf();

-- Run k6 load test, then:
-- tail -f /var/log/postgresql/postgresql.log
-- Every slow query appears with SQL + execution time + caller
-- This is how you find every performance problem in your app
```

Seed 1M rows. Run the load test. Watch your server choke. Fix the missing index. Run the test again. See the difference. This single exercise is worth more than a year of tutorial projects.

— 17 —

Quick Reference — Rules to Live By

■ **Validate server-side** — Frontend validation is UX only. Backend must reject bad input independently. Return field-level messages.

■ **Generic auth errors** — Never say 'email not found' or 'wrong password'. Always 'Invalid credentials'. Never confirm existence.

■ **Check ownership** — Every query on user-owned data must include `userId` (and `tenantId` in SaaS). ID alone = IDOR.

■ **Index from day one** — Don't wait for 10K rows. Index every `WHERE`, `JOIN`, `ORDER BY` column. `UNIQUE` index for unique fields.

■ **Fix N+1 queries** — Never query inside a loop. Use `JOIN`, eager loading (`include/populate`), or raw SQL.

■ **Always paginate** — No endpoint returns unbounded rows. Cursor-based over `OFFSET`. Hard server-side `LIMIT` cap.

■ **Parallel independent calls** — If A doesn't need B's result: `Promise.all()`. Sequential is wasted latency.

■ **bcrypt for passwords** — Min cost 10. Never MD5, SHA1, plain text. Use environment variables for all secrets.

■ **Loading states** — One action = one API call. Disable buttons during in-flight requests. AbortController in useEffect.

■ **Return minimum data** — Email, phone, password never leave the server in public responses. Select only what's needed.

■ **SELECT FOR UPDATE** — Credit deduction, inventory, reservations — always use atomic updates or row locks.

■ **Object storage always** — Never store uploads on your app server. S3, DO Spaces, BunnyCDN, Cloudflare R2.

■ **Proxy in front** — App server never directly exposed. Cloudflare + ALB/Nginx in front. DB on private subnet.

■ **Error monitoring** — Bugsnag or Sentry + Slack alerts. You should know about crashes before users do.

■ **Rate limit in layers** — Cloudflare/AWS WAF at the edge. express-rate-limit in app. Both. Always.

■ **Simulate scale** — Seed 1M rows + k6 load test + slow query log. Do it before shipping, not after.

By **Sadek Hossain**

Founder & CEO, EzyCourse

f facebook.com/sadek.hkm.hossain

in linkedin.com/in/hkm-hossain

For team onboarding, code reviews, and developer community sharing.